November 6, 2026 at 22:52

1. Introduction. Section 7.2.2.2 has a little parable about learning a Boolean function. Table 2 lists thirty-two points $x = x_1 \dots x_{20}$, sixteen where a hidden function is 1 and sixteen where it is 0, and a `SAT` solver soon finds a four-term DNF, Eq. (27), that fits every one of them. Exercise 56 then asks for a fit that depends on only five of the twenty variables, and exercise 57, the one this program is about, goes further:

> **57.** [**29**] Combining the previous exercise with the methods of Section 7.1.2, exhibit a function f for Table 2 that can be evaluated with only six Boolean operations(!).

The answer recalls from answer 56 that there are twelve sets of five variables on which Table 2 can be defined consistently, prints the twelve resulting truth tables with their don't-cares, and says that "the tenth of these yields"

$$f(x) = ((x_8 \oplus (x_9 \vee x_{10})) \vee ((x_6 \vee x_{12}) \oplus \bar{x}_{10})) \oplus x_{12}.$$

I wrote this on 16 September 2026, as the first of a series of careful readings of the §7.2.2.2 exercises that Knuth's news page asks readers to check. It checks every statement the answer makes, from the transcription of Table 2 to the six operations, and then asks the question the exclamation mark invites: could it be done with five? It cannot, and the program shows that twice, once by exhaustive search and once with the `sat` package of the directory above, which is a Go rendering of Knuth's `SAT13`. Along the way it finds that the tenth table is one of five that allow six operations.

2. The program is a list of modes, one per claim.

```
package main
import (
    "context"
    "flag"
    "fmt"
    "math/big"
    "math/bits"
    "math/rand/v2"
    "slices"
    "strings"
    "sync"
    "time"
    "github.com/sjnam/sat"
)
⟨ Types 8 ⟩
⟨ Global variables 4 ⟩
⟨ Functions 5 ⟩
func main() {
    ⟨ Read the command line 3 ⟩
    ⟨ Read Table 2 10 ⟩
    ⟨ Find the covering sets 14 ⟩
    if on("table2") {
        ⟨ Check Table 2 against π and e 11 ⟩
    }
    if on("covers") {
        ⟨ Report the covering sets 18 ⟩
    }
    if on("tables") {
        ⟨ Compare the twelve truth tables with the answer 21 ⟩
    }
    if on("formula") {
        ⟨ Check the formula and count its operations 28 ⟩
    }
    if on("exhaustive") {
        ⟨ Rule out five steps by exhaustive search 35 ⟩
    }
    if on("sat") {
        ⟨ Rule out five steps, and find six, with the SAT solver 51 ⟩
    }
    if on("selftest") {
        ⟨ Test the two searches against each other 55 ⟩
    }
    if mode ≡ "direct" {
        ⟨ Try answer 477 on all twenty variables 54 ⟩
    }
}
```

3. Every mode but `direct` runs under `-mode all`, which takes about seven minutes, most of it spent in the last part of the `sat` mode. The mode `direct` is there to show why the argument of (iv) below is needed.

⟨ Read the command line 3 ⟩ ≡

```
flag.StringVar(&mode, "mode", "all", "table2,␣covers,␣tables,␣formula,␣" +
    "exhaustive,␣sat,␣selftest,␣all,␣or␣direct")
trials := flag.Int("trials", 1000, "random␣instances␣for␣selftest")
maxSteps := flag.Int("r", 4, "the␣most␣steps␣to␣try␣in␣direct␣mode")
flag.Parse()
```

This code is used in section 2.

4. ⟨ Global variables 4 ⟩ ≡

```
var mode string
```

This code is also defined in sections 6, 7, 9, 15, 20, 27.

This code is used in section 2.

5. ⟨ Functions 5 ⟩ ≡

```
func on(m string) bool { return mode ≡ m ∨ mode ≡ "all" }
```

This code is also defined in sections 13, 16, 17, 19, 23, 25, 26, 31, 32, 33, 34, 36, 40, 41, 43.

This code is used in section 2.

6. Table 2. Here is the table, copied from page 199. Each row is $x_1x_2\ldots x_{20}$.

⟨ Global variables 4 ⟩ +≡

```
var leftRows = [16]string{   // the cases where f(x) = 1
    "11001001000011111101", "10101010001000100001",
    "01101000110000100011", "01001100010011000110",
    "01100010100010111000", "00001101110000011100",
    "11010001001010010000", "00100100111000001000",
    "10001010011001111100", "11000111010000000010",
    "00001011101111101010", "01100011101100010011",
    "10011011001000100101", "00010100101000001000",
    "01111001100011100011", "01000000010011011101",
}
```

7. ⟨ Global variables 4 ⟩ +≡

```
var rightRows = [16]string{   // the cases where f(x) = 0
    "10101101111110000101", "01000101100010100010",
    "10111011010010101001", "10101010111111011100",
    "01010110001000000010", "01110011110100111100",
    "11110001110110001011", "10011100010110000011",
    "11001110001011010011", "01101001010110101001",
    "11100001001101100100", "00010001010001100100",
    "00110011111110111100", "11001001001110011101",
    "11001110001001001001", "10110011111011111001",
}
```

8. A point packs x_j into bit $j-1$ of a word.

⟨ Types 8 ⟩ ≡

```
type point struct {
    x uint32   // bit j − 1 is x_j
    val bool   // the value of f there
}
```

This code is also defined in sections 24, 39.

This code is used in section 2.

9. ⟨ Global variables 4 ⟩ +≡

```
var points []point   // the sixteen ones, then the sixteen zeros
```

10. ⟨ Read Table 2 10 ⟩ ≡

```
for side, rows := range [2][16]string { leftRows, rightRows }{
    for _, s := range rows {
        var x uint32
        for j := 0; j < 20; j++ {
            if s[j] ≡ '1' {
                x |= 1 ≪ j
            }
        }
        points = append(points, point{x, side ≡ 0})
    }
}
```

This code is used in section 2.

11. A transcription slip would spoil everything downstream, so it is worth having a second source. Answer 53 gives one: the left column is $\pi/4$ in binary, twenty bits at a time, and the right column is $e/4$. Machin's formula $\pi/4 = 4\arctan\frac{1}{5} - \arctan\frac{1}{239}$ and the series for e, carried to 400 bits, are more than enough for 320.

⟨ Check Table 2 against π and e 11 ⟩ ≡

```
const prec = 400
quarterPi := new(big.Float).SetPrec(prec).Mul(arctanInv(5, prec),
    big.NewFloat(4))
quarterPi.Sub(quarterPi, arctanInv(239, prec))
e := new(big.Float).SetPrec(prec)
term := new(big.Float).SetPrec(prec).SetInt64(1)
for k := int64(1); k ≤ 120; k++ {
    e.Add(e, term)
    term.Quo(term, new(big.Float).SetPrec(prec).SetInt64(k))
}
e.Quo(e, big.NewFloat(4))
⟨ Compare the binary expansions with the two columns 12 ⟩
```

This code is used in section 2.

12. ⟨ Compare the binary expansions with the two columns 12 ⟩ ≡

```
for side, v := range []*big.Float { quarterPi, e }{
    scaled := new(big.Float).SetPrec(prec).SetMantExp(v, 320)
    n, _ := scaled.Int(Λ)
    s := fmt.Sprintf("%0320b", n)
    rows, agree := [2][16]string{leftRows, rightRows}[side], 0
    for k := range 16 {
        if s[20*k:20*k+20] ≡ rows[k] {
            agree++
        }
    }
    fmt.Printf("Table 2, %s column: %d of 16 rows are %s in binary\n",
        []string{"left", "right"}[side], agree,
        []string{"pi/4", "e/4"}[side])
}
```

This code is used in section 11.

13. The series $\arctan(1/x) = \sum_k (-1)^k/((2k+1)x^{2k+1})$ loses about $2\lg x$ bits a term, so two hundred terms do for $x \geq 5$.

⟨ Functions 5 ⟩ +≡

```
func arctanInv(x int64, prec uint) *big.Float {
    sum := new(big.Float).SetPrec(prec)
    xx := new(big.Float).SetPrec(prec).SetInt64(x)
    pow := new(big.Float).SetPrec(prec).Quo(
        new(big.Float).SetPrec(prec).SetInt64(1), xx)
    xx.Mul(xx, xx)
    for k := int64(0); k < 200; k++ {
        t := new(big.Float).SetPrec(prec).Quo(pow,
            new(big.Float).SetPrec(prec).SetInt64(2*k + 1))
        if k%2 == 0 {
            sum.Add(sum, t)
        } else {
            sum.Sub(sum, t)
        }
        pow.Quo(pow, xx)
    }
    return sum
}
```

14. Covering sets. A set S of variables determines f consistently exactly when no point on the left agrees with a point on the right in all the positions of S. With u_k and v_l for the rows, as in answer 56, that says S meets every one of the 256 differences $u_k \oplus v_l$. There are only 2^{20} sets, so I simply try them all, counting by size and keeping the smallest.

⟨ Find the covering sets 14 ⟩ ≡

```
for _, p := range points[:16] {
    for _, q := range points[16:] {
        diffs = append(diffs, p.x ⊕ q.x)
    }
}
for m := uint32(0); m < 1 ≪ 20; m++ {
    if covering(m) {
        gen[bits.OnesCount32(m)]++
        if bits.OnesCount32(m) ≡ 5 {
            fives = append(fives, m)
        }
    }
}
slices.SortFunc(fives, func(a, b uint32) int {
    return slices.Compare(members(a), members(b))
})
```

This code is used in section 2.

15. ⟨ Global variables 4 ⟩ +≡

```
var (
    diffs []uint32    // the 256 vectors u_k ⊕ v_l
    gen [21]int    // how many covering sets there are of each size
    fives []uint32    // the covering sets of size five, in lexicographic order
)
```

16. ⟨ Functions 5 ⟩ +≡

```
func covering(m uint32) bool {
    for _, d := range diffs {
        if d & m ≡ 0 {
            return false
        }
    }
    return true
}
```

17. A set is a bit mask; *members* lists its variables, numbered from 0.

⟨ Functions 5 ⟩ +≡

```
func members(m uint32) []int {
    var js []int
    for j := 0; j < 20; j++ {
        if m >> j & 1 == 1 {
            js = append(js, j)
        }
    }
    return js
}
func setName(m uint32) string {
    var s []string
    for _, j := range members(m) {
        s = append(s, fmt.Sprint(j + 1))
    }
    return "{" + strings.Join(s, ",") + "}"
}
```

18. Answer 56 lists the twelve sets and gives the generating function $12z^5+994z^6+13503z^7+\cdots+20z^{19}+z^{20}$.

⟨ Report the covering sets 18 ⟩ ≡

```
answer56 := "{1,4,15,17,20}␣{1,10,15,17,20}␣{1,15,17,18,20}␣" +
    "{4,6,7,10,12}␣{4,6,9,10,12}␣{4,6,10,12,19}␣{4,10,12,15,19}␣" +
    "{5,7,11,12,15}␣{6,7,8,10,12}␣{6,8,9,10,12}␣{7,10,12,15,20}␣" +
    "{8,15,17,18,20}"
var names []string
for _, m := range fives {
    names = append(names, setName(m))
}
fmt.Println("covering␣sets␣by␣size:", gen)
fmt.Printf("the␣%d␣five-sets␣agree␣with␣answer␣56:␣%v\n", len(fives),
    strings.Join(names, "␣") ≡ answer56)
fmt.Println("␣␣and␣so␣do␣the␣coefficients␣12,␣994,␣13503,␣...,␣20,␣1:",
    gen[5] ≡ 12 ∧ gen[6] ≡ 994 ∧ gen[7] ≡ 13503 ∧
        gen[19] ≡ 20 ∧ gen[20] ≡ 1)
```

This code is used in section 2.

19. The twelve truth tables. Section 7.1.2 writes the truth table of $g(y_1, \ldots, y_w)$ as the 2^w bits $g(0, \ldots, 0) \ldots g(1, \ldots, 1)$, with y_1 the most significant. So restricting a point to a set S gives a position in the table of S, the first variable of S leading, and *ttIndex* computes it.

⟨ Functions 5 ⟩ +≡

```
func ttIndex(x, m uint32) int {
    t := 0
    for _, j := range members(m) {
        t = 2 * t + int(x >> j & 1)
    }
    return t
}
```

20. The answer sets the twelve tables in two columns of six, each table being four groups of eight bits. I read them down the left column first.

⟨ Global variables 4 ⟩ +≡

```
var printedLeft = [6]string{
    "11110110␣0*1*010*␣10000111␣10*0*1*0",
    "011*011*␣1*110100␣10*001*1␣1000**10",
    "011*1*11␣010*100*␣10*0*000␣*101*011",
    "10101110␣0*100*1*␣1*001*00␣1**00***",
    "10101110␣0*1*0*10␣1*0*1*00␣0**01***",
    "1*01110*␣00**110*␣11**0*00␣10*****0",
}
var printedRight = [6]string{
    "00100101␣11110*0*␣1011****␣**0**00*",
    "100*1**0␣11*00010␣1100**0*␣*0**0101",
    "**1*1000␣1*101100␣1*100*10␣0*****1*",
    "1*1*1*10␣10001100␣0*101*1*␣**1*0*10",
    "1*01*00*␣1101*0*0␣0011*11*␣1*100*0*",
    "001*1001␣*1**1*1*␣11*0*010␣01011001",
}
```

21. The comparison is done in both reading orders, down the columns and across the rows, because only one of them can be what was meant.

⟨ Compare the twelve truth tables with the answer 21 ⟩ ≡

```
var down, across []string
for _, col := range [2][6]string { printedLeft, printedRight }{
    for _, s := range col {
        down = append(down, strings.ReplaceAll(s, "␣", ""))
    }
}
for r := range 6 {
    across = append(across, down[r], down[6 + r])
}
downOK, acrossOK := 0, 0
for i, m := range fives {
    ⟨ Build the table of m and compare it with the printed ones 22 ⟩
}
fmt.Printf("tables␣that␣agree,␣reading␣down␣the␣columns:␣%d␣of␣12\n", downOK)
fmt.Printf("tables␣that␣agree,␣reading␣across␣the␣rows:␣␣%d␣of␣12\n",
    acrossOK)
```

This code is used in section 2.

22. ⟨ Build the table of m and compare it with the printed ones 22 ⟩ ≡

```
tt := []byte(strings.Repeat("*", 32))
for _, p := range points {
    tt[ttIndex(p.x, m)] = "01"[b2i(p.val)]
}
if string(tt) ≡ down[i] {
    downOK ++
}
if string(tt) ≡ across[i] {
    acrossOK ++
}
fmt.Printf("%2d␣%-16s␣%s\n", i + 1, setName(m), tt)
```

This code is used in section 21.

23. ⟨ Functions 5 ⟩ +≡

```
func b2i(b bool) int {
    if b {
        return 1
    }
    return 0
}
```

24. Chains. A Boolean chain has n inputs and r steps $x_i = x_j \circ_i x_k$ with $j < k < i$; I number the nodes from 0. An operation is a four-bit truth table, bit $2a + b$ being $a \circ b$, so `#8` is $\wedge$, `#6` is $\oplus$, and `#e` is $\vee$. A chain may end with a complement.

⟨ Types 8 ⟩ +≡

```
type step struct {
    j, k int
    op uint8
}
type chain struct {
    n int
    steps []step
    comp bool   // is the output complemented?
}
```

25. Input q of a chain is bit q of *row*.

⟨ Functions 5 ⟩ +≡

```
func (ch *chain) eval(row uint32) bool {
    v := make([]bool, ch.n, ch.n + len(ch.steps))
    for q := range ch.n {
        v[q] = row >> q & 1 == 1
    }
    for _, s := range ch.steps {
        a, b := b2i(v[s.j]), b2i(v[s.k])
        v = append(v, s.op >> (2 * a + b) & 1 == 1)
    }
    return v[len(v) - 1] != ch.comp
}
```

26. Steps are printed as g_1, g_2, ..., and inputs by the names given.

⟨ Functions 5 ⟩ +≡

```
var opForm = [16]string{"0", "~(a|b)", "~a&b", "~a", "a&~b", "~b", "a^b",
    "~(a&b)", "a&b", "~(a^b)", "b", "~a|b", "a", "a|~b", "a|b", "1"}
func (ch *chain) format(names []string) string {
    name := func(i int) string {
        if i < ch.n {
            return names[i]
        }
        return fmt.Sprintf("g%d", i - ch.n + 1)
    }
    var s []string
    for i, st := range ch.steps {
        r := strings.NewReplacer("a", name(st.j), "b", name(st.k))
        s = append(s, name(ch.n + i) + " = " + r.Replace(opForm[st.op]))
    }
    return strings.Join(s, "; ")
}
```

27. The formula. Written as a chain on all twenty variables, the answer's formula is six steps:

$$g_1 = x_9 \vee x_{10}, \quad g_2 = x_8 \oplus g_1, \quad g_3 = x_6 \vee x_{12},$$
$$g_4 = x_{10} \equiv g_3, \quad g_5 = g_2 \vee g_4, \quad g_6 = g_5 \oplus x_{12}.$$

The fourth, $(x_6 \vee x_{12}) \oplus \bar{x}_{10}$, is one operation, since $\equiv$ is one of the sixteen. Section 7.1.2 prefers *normal* chains, whose operations all take $(0, 0)$ to 0, and the conversion costs nothing: $g'_4 = x_{10} \oplus g_3$, $g'_5 = \bar{g}_2 \wedge g'_4 = \bar{g}_5$, and $g'_6 = g'_5 \oplus x_{12} = \bar{g}_6$, with the output complemented.

⟨ Global variables 4 ⟩ +≡

```
var bookChain = chain{n: 20, steps: []step{
    {8, 9, #e}, {7, 20, #6}, {5, 11, #e},
    {9, 22, #9}, {21, 23, #e}, {11, 24, #6}}}
var normalChain = chain{n: 20, comp: true, steps: []step{
    {8, 9, #e}, {7, 20, #6}, {5, 11, #e},
    {9, 22, #6}, {21, 23, #2}, {11, 24, #6}}}
```

28. Three renderings, then: the formula as printed, typed into Go, and the two chains. All must agree with Table 2 everywhere.

⟨ Check the formula and count its operations 28 ⟩ ≡

```
x := func(x uint32, j int) bool { return x >> (j - 1) & 1 == 1 }
agree := [3]int{}
for _, p := range points {
    f := ((x(p.x, 8) != (x(p.x, 9) || x(p.x, 10))) ||
        ((x(p.x, 6) || x(p.x, 12)) != !x(p.x, 10))) != x(p.x, 12)
    for i, v := range []bool { f, bookChain.eval(p.x), normalChain.eval(p.x) }{
        if v == p.val {
            agree[i]++
        }
    }
}
var xnames []string
for j := 1; j <= 20; j++ {
    xnames = append(xnames, fmt.Sprintf("x%d", j))
}
fmt.Printf("formula, chain, normal chain agree with Table 2 at %v points\n",
    agree)
fmt.Printf("the chain has %d steps: %s\n", len(bookChain.steps),
    bookChain.format(xnames))
fmt.Printf("normal form, output complemented: %s\n",
    normalChain.format(xnames))
⟨ Show how the formula fills in the tenth table 29 ⟩
```

This code is used in section 2.

29. The formula's own truth table on $(x_6, x_8, x_9, x_{10}, x_{12})$ must agree with the tenth table wherever that table is not a star.

⟨ Show how the formula fills in the tenth table 29 ⟩ ≡

```
m := fives[9]
var full []byte
for t := 0; t < 32; t++ {
    var row uint32
    for q, j := range members(m) {
        row |= uint32(t >> (4 - q) & 1) << j
    }
    full = append(full, "01"[b2i(bookChain.eval(row))])
}
tenth, ok := strings.ReplaceAll(printedRight[3], "␣", ""), true
for t := range 32 {
    ok = ok ∧ (tenth[t] ≡ '*' ∨ tenth[t] ≡ full[t])
}
fmt.Printf("on␣%s␣the␣formula's␣table␣is␣%s\n", setName(m), full)
fmt.Printf("␣␣the␣printed␣tenth␣table␣is␣␣␣␣␣␣␣%s;␣consistent:␣%v\n",
    tenth, ok)
```

This code is used in section 28.

30. Why five steps cannot do: the argument. Suppose some chain of at most five steps agrees with Table 2, and take a shortest one, of r steps. By Section 7.1.2 we may take it normal, with a complement at the end if need be. Then:

(i) $r > 0$, because no covering set has fewer than five elements.

(ii) Every step but the last is used by a later step, or it could be deleted; no step uses a trivial operation (0, a, or b), or it could be bypassed.

(iii) The $2r$ operand slots include at least $r - 1$ references to steps, hence at most $r + 1$ to inputs. The inputs that occur form a covering set, since the output depends on nothing else.

(iv) So either the chain reads at most five variables—and then exactly five, one of the twelve sets—or $r = 5$ and it reads six. In that second case all ten slots are spoken for: each variable is read once and each step is used once. The chain is a *read-once formula*.

Both searches below rest on (iv). The exhaustive one looks at every chain of at most five steps on each of the twelve sets, and at every read-once formula on each of the 994 covering sets of six; the `SAT` one sets up the clauses of answer 477 for the same two families. They share nothing else.

31. Exhaustive search. On five variables a function is a 32-bit truth table, on six a 64-bit one; *varTable* gives the table of variable q of w, the first leading, which is the convention of *ttIndex*. A covering set turns Table 2 into a partial table: *care* marks the positions that are specified and *want* holds their values.

⟨ Functions 5 ⟩ +≡

```
func varTable(q, w int) uint64 {
    var v uint64
    for t := 0; t < 1 ≪ w; t++ {
        if t ≫ (w − 1 − q) & 1 ≡ 1 {
            v |= 1 ≪ t
        }
    }
    return v
}
func target(m uint32)(care, want uint64) {
    for _, p := range points {
        t := ttIndex(p.x, m)
        care |= 1 ≪ t
        if p.val {
            want |= 1 ≪ t
        }
    }
    return
}
```

32. A table v fits when it agrees with *want* on all of *care*, or disagrees on all of it; the second case is the complemented output.

⟨ Functions 5 ⟩ +≡

```
var normalOps = [5]uint8{#8, #4, #2, #6, #e}
func apply(op uint8, a, b uint64)(v uint64) {
    for i, w := range [4]uint64 {⊕a ⊘ b, ⊕a & b, a ⊘ b, a & b }{
        if op ≫ i & 1 ≡ 1 {
            v |= w
        }
    }
    return
}
func fits(v, care, want uint64) bool {
    d := (v ⊕ want) & care
    return d ≡ 0 ∨ d ≡ care
}
```

33. Here is the search on five variables. It returns the fewest steps of any chain of at most *limit* steps that fits, or −1. It deepens one step at a time, so an easy target is found quickly. At each depth the last step is not built but only tested, all pairs and all five operations; the earlier steps are built, except that a step whose table is already present is skipped, since it could be deleted.

⟨ Functions 5 ⟩ +≡

```
func cheapest(care, want uint64, limit int) int {
    nodes := make([]uint64, 0, 5 + limit)
    for q := range 5 {
        nodes = append(nodes, varTable(q, 5))
        if fits(nodes[q], care, want) {
            return 0
        }
    }
    for depth := 1; depth ≤ limit; depth++ {
        if extend(nodes, depth − 1, care, want) {
            return depth
        }
    }
    return −1
}
```

34. ⟨ Functions 5 ⟩ +≡

```
func extend(nodes []uint64, more int, care, want uint64) bool {
    L := len(nodes)
    for k := 1; k < L; k++ {
        for j := 0; j < k; j++ {
            for _, op := range normalOps {
                v := apply(op, nodes[j], nodes[k])
                if more ≡ 0 {
                    if fits(v, care, want) {
                        return true
                    }
                } else if ¬slices.Contains(nodes, v) ∧
                extend(append(nodes, v), more − 1, care, want) {
                    return true
                }
            }
        }
    }
    return false
}
```

35. The twelve sets are independent, so they run side by side.

⟨ Rule out five steps by exhaustive search 35 ⟩ ≡

```
start := time.Now()
cost := make([]int, len(fives))
var wg sync.WaitGroup
for i, m := range fives {
    wg.Add(1)
    go func() {
        defer wg.Done()
        care, want := target(m)
        cost[i] = cheapest(care, want, 5)
    }()
}
wg.Wait()
fmt.Printf("fewest␣steps␣of␣at␣most␣five␣on␣the␣twelve␣sets␣(-1␣if␣none):␣" +
    "%v,␣%.1fs\n", cost, time.Since(start).Seconds())
⟨ Rule out read-once formulas on six variables 38 ⟩
```

This code is used in section 2.

36. A normal read-once function on a set S of variables is a single variable, or $a \circ b$ with a and b normal read-once functions on the two parts of a split of S and $\circ$ one of the five nontrivial normal operations. The smallest variable of S goes to the first part; the order of the operands does not matter, since $a \wedge \bar{b}$ and $\bar{a} \wedge b$ are both there.

⟨ Functions 5 ⟩ +≡

```
func readOnce(S uint, memo map[uint]map[uint64]bool) map[uint64]bool {
    if r, ok := memo[S]; ok {
        return r
    }
    r := map[uint64]bool{}
    if bits.OnesCount(S) ≡ 1 {
        r[varTable(bits.TrailingZeros(S), 6)] = true
    } else {
        low := S & -S
        rest := S ⊘ low
        for A := rest; ; A = (A - 1) & rest {
            ⟨ Combine the read-once functions on low ∪ A and on the rest 37 ⟩
            if A ≡ 0 {
                break
            }
        }
    }
    memo[S] = r
    return r
}
```

37. ⟨ Combine the read-once functions on $low \cup A$ and on the rest 37 ⟩ ≡

```
if B := S ⊘ (low | A); B ≠ 0 {
    for a := range readOnce(low | A, memo) {
        for b := range readOnce(B, memo) {
            for _, op := range normalOps {
                r[apply(op, a, b)] = true
            }
        }
    }
}
```

This code is used in section 36.

38. ⟨ Rule out read-once formulas on six variables 38 ⟩ ≡

```
start = time.Now()
all := readOnce(63, map[uint]map[uint64]bool{})
sixes, matches := 0, 0
for m := uint32(0); m < 1 ≪ 20; m++ {
    if bits.OnesCount32(m) ≡ 6 ∧ covering(m) {
        sixes++
        care, want := target(m)
        for v := range all {
            if fits(v, care, want) {
                matches++
            }
        }
    }
}
fmt.Printf("%d read-once functions of six variables, %d covering six-sets, " +
    "%d fits, %.1fs\n", len(all), sixes, matches, time.Since(start).Seconds())
```

This code is used in section 35.

39. The SAT route. An instance is what a chain has to match: n inputs, and a list of rows, input q being bit q of the row, each with its wanted value.

⟨ Types 8 ⟩ +≡

```
type instance struct {
    n int
    rows []uint32
    want []bool
}
```

40. Restricting Table 2 to a covering set gives such an instance, input q being the qth smallest variable of the set. Points that restrict to the same row are kept once.

⟨ Functions 5 ⟩ +≡

```
func instanceOn(m uint32) instance {
    js := members(m)
    in := instance{n: len(js)}
    seen := map[uint32]bool{}
    for _, p := range points {
        var row uint32
        for q, j := range js {
            row |= (p.x >> j & 1) << q
        }
        if !seen[row] {
            seen[row] = true
            in.rows = append(in.rows, row)
            in.want = append(in.want, p.val)
        }
    }
    return in
}
```

41. Answer 477 uses a variable x_{it} for the value of step i at row t, a variable s_{ijk} saying that step i has operands j and k, and variables f_{ipq} for its operation at $(p, q) \neq (0, 0)$. I add one more, *comp*, for a complemented output, because a partial function need not be matched by a normal function itself. Nodes are numbered from 0 here too.

```
⟨ Functions 5 ⟩ +≡
  func solveChain(in instance, r int)(sat.Status, *chain, sat.Stats) {
    s := sat.New()
    n, P, N := in.n, len(in.rows), in.n + r
    ⟨ Create the variables of answer 477 42 ⟩
    for i := n; i < N; i++ {
      var some []sat.Lit
      for k := 1; k < i; k++ {
        for j := 0; j < k; j++ {
          some = append(some, sel[i][j][k])
          ⟨ Emit the main clauses for s_ijk 44 ⟩
        }
      }
      s.AddClause(some...)
    }
    ⟨ Emit the optional clauses of answers 477 and 478 47 ⟩
    ⟨ Emit the output clauses 46 ⟩
    st, err := s.Solve(context.Background())
    if err ≠ Λ {
      panic(err)
    }
    if st ≠ sat.Sat {
      return st, Λ, s.Stats()
    }
    ⟨ Read the chain off the solution 50 ⟩
    return st, ch, s.Stats()
  }
```

42. ⟨ Create the variables of answer 477 42 ⟩ ≡

```
x := make([][]sat.Lit, N)
f := make([][4]sat.Lit, N)
sel := make([][][]sat.Lit, N)
for i := n; i < N; i++ {
    x[i] = make([]sat.Lit, P)
    for t := range P {
        x[i][t] = s.NewVar()
    }
    for pq := 1; pq < 4; pq++ {
        f[i][pq] = s.NewVar()
    }
    sel[i] = make([][]sat.Lit, i)
    for j := range i {
        sel[i][j] = make([]sat.Lit, i)
        for k := j + 1; k < i; k++ {
            sel[i][j][k] = s.NewVar()
        }
    }
}
comp := s.NewVar()
```

This code is used in section 41.

43. The literal $(x \oplus a)$ of answer 477 is true when $x \neq a$.

⟨ Functions 5 ⟩ +≡

```
func differs(l sat.Lit, a int) sat.Lit {
    if a ≡ 1 {
        return l.Not()
    }
    return l
}
```

44. The main clauses are $(\bar{s}_{ijk} \vee (x_{it} \oplus a) \vee (x_{jt} \oplus b) \vee (x_{kt} \oplus c) \vee (f_{ibc} \oplus \bar{a}))$. When $b = c = 0$ the last literal is false, and when also $a = 0$ the clause is dropped. When an operand is an input its value is known: a true literal drops the clause and a false one is left out, just as the answer says.

⟨ Emit the main clauses for s_{ijk} 44 ⟩ ≡

```
for t := range P {
    for a := range 2 {
        for bc := range 4 {
            if a ≡ 0 ∧ bc ≡ 0 {
                continue
            }
            cl := []sat.Lit{sel[i][j][k].Not(), differs(x[i][t], a)}
            ⟨ Add the operand literals, or skip the clause 45 ⟩
            if bc ≠ 0 {
                cl = append(cl, differs(f[i][bc], 1 − a))
            }
            s.AddClause(cl...)
        }
    }
}
```

This code is used in section 41.

45. ⟨ Add the operand literals, or skip the clause 45 ⟩ ≡

```
satisfied := false
for _, o := range [2][2]int {{j, bc ≫ 1}, {k, bc & 1} }{
    if o[0] ≥ n {
        cl = append(cl, differs(x[o[0]][t], o[1]))
    } else if int(in.rows[t] ≫ o[0] & 1) ≠ o[1] {
        satisfied = true
    }
}
if satisfied {
    continue
}
```

This code is used in section 44.

46. ⟨ Emit the output clauses 46 ⟩ ≡

```
for t := range P {
    if in.want[t] {
        s.AddClause(x[N − 1][t], comp)
        s.AddClause(x[N − 1][t].Not(), comp.Not())
    } else {
        s.AddClause(x[N − 1][t], comp.Not())
        s.AddClause(x[N − 1][t].Not(), comp)
    }
}
```

This code is used in section 41.

47. Answers 477 and 478 add clauses that are not needed for correctness but cut the search down enormously; without them the six-step question on a single five-set did not finish in a minute. They must not cut off every shortest chain, and they don't. Take a shortest chain, and among those one whose operand indices have the smallest sum. By (ii) it uses every step and no trivial operation. It never has a step i' with operands j and i where step i already uses j: that step is a function of j and i's other operand k, so it can be recomputed from j and k, which lowers the sum. Finally, to put the operand pairs in colex order whenever step $i+1$ does not use step i, renumber the steps greedily, always taking next the step with the colex-least pair among those whose operands are already placed: if step $i+1$ does not use step i it was available when step i was chosen. Renumbering changes none of the other properties, which do not depend on the numbering.

⟨ Emit the optional clauses of answers 477 and 478 47 ⟩ ≡

```
for i := n; i < N; i++ {
    s.AddClause(f[i][1], f[i][2], f[i][3])
    s.AddClause(f[i][1], f[i][2].Not(), f[i][3].Not())
    s.AddClause(f[i][1].Not(), f[i][2], f[i][3].Not())
    if i < N − 1 {
        ⟨ Step i is used by a later step 48 ⟩
    }
    for k := 1; k < i; k++ {
        for j := 0; j < k; j++ {
            ⟨ Forbid reapplied operands and colex inversions after s_ijk 49 ⟩
        }
    }
}
```

This code is used in section 41.

48. ⟨ Step i is used by a later step 48 ⟩ ≡

```
var used []sat.Lit
for ii := i + 1; ii < N; ii++ {
    for j := 0; j < i; j++ {
        used = append(used, sel[ii][j][i])
    }
    for k := i + 1; k < ii; k++ {
        used = append(used, sel[ii][i][k])
    }
}
s.AddClause(used...)
```

This code is used in section 47.

49. ⟨ Forbid reapplied operands and colex inversions after s_{ijk} 49 ⟩ ≡

```
for ii := i + 1; ii < N; ii++ {
    s.AddClause(sel[i][j][k].Not(), sel[ii][j][i].Not())
    s.AddClause(sel[i][j][k].Not(), sel[ii][k][i].Not())
}
if i + 1 < N {
    for kk := 1; kk < i ∧ kk ≤ k; kk++ {
        for jj := 0; jj < kk ∧ (kk < k ∨ jj < j); jj++ {
            s.AddClause(sel[i][j][k].Not(), sel[i + 1][jj][kk].Not())
        }
    }
}
```

This code is used in section 47.

50. Any selected pair will do: every selected pair's clauses hold.

⟨ Read the chain off the solution 50 ⟩ ≡

```
ch := &chain{n: n, comp: s.Value(comp)}
for i := n; i < N; i++ {
    st := step{j: −1}
    for k := 1; k < i ∧ st.j < 0; k++ {
        for j := 0; j < k ∧ st.j < 0; j++ {
            if s.Value(sel[i][j][k]) {
                st.j, st.k = j, k
            }
        }
    }
    for pq := 1; pq < 4; pq++ {
        if s.Value(f[i][pq]) {
            st.op |= 1 ≪ pq
        }
    }
    ch.steps = append(ch.steps, st)
}
```

This code is used in section 41.

51. On each of the twelve sets I ask for chains of $r = 1, 2, \ldots, 6$ steps. By (i) and (ii) the clauses are exact for $r \geq 1$, so five `UNSAT`s rule out five steps; the sixth run says whether six will do, and a chain it finds is checked against all thirty-two points of Table 2, not only against the clauses.

```
⟨ Rule out five steps, and find six, with the SAT solver 51 ⟩ ≡
    start := time.Now()
    var mems uint64
    for i, m := range fives {
        in := instanceOn(m)
        var names []string
        for _, j := range members(m) {
            names = append(names, fmt.Sprintf("x%d", j + 1))
        }
        var sts []sat.Status
        found := ""
        for r := 1; r ≤ 6; r++ {
            st, ch, stats := solveChain(in, r)
            mems += stats.Mems
            sts = append(sts, st)
            ⟨ If a chain was found, check it on Table 2 and describe it 52 ⟩
        }
        fmt.Printf("%2d %-16s r=1..6: %v\n%s", i + 1, setName(m), sts, found)
    }
    fmt.Printf("twelve sets: %d megamems, %.1fs\n", mems/1000000,
        time.Since(start).Seconds())
    ⟨ Rule out five steps on the six-sets with the SAT solver 53 ⟩
```

This code is used in section 2.

52.

```
⟨ If a chain was found, check it on Table 2 and describe it 52 ⟩ ≡
    if ch ≠ Λ {
        agree := 0
        for _, p := range points {
            var row uint32
            for q, j := range members(m) {
                row |= (p.x ≫ j & 1) ≪ q
            }
            if ch.eval(row) ≡ p.val {
                agree++
            }
        }
        out := ""
        if ch.comp {
            out = ", output complemented"
        }
        found = fmt.Sprintf("   agrees at %d points: %s%s\n", agree,
            ch.format(names), out)
    }
```

This code is used in section 51.

53. By (iv) the six-sets only need $r = 5$. This is the slow part: 994 instances, each an `UNSAT` proof, and together almost six minutes.

⟨ Rule out five steps on the six-sets with the `SAT` solver 53 ⟩ ≡

```
start, mems = time.Now(), 0
sixes, satisfiable := 0, 0
for m := uint32(0); m < 1 ≪ 20; m++ {
    if bits.OnesCount32(m) ≡ 6 ∧ covering(m) {
        sixes++
        st, _, stats := solveChain(instanceOn(m), 5)
        mems += stats.Mems
        if st ≠ sat.Unsat {
            satisfiable++
        }
    }
}
fmt.Printf("%d covering six-sets, %d not UNSAT with five steps: " +
    "%d megamems, %.1fs\n", sixes, satisfiable, mems/1000000,
    time.Since(start).Seconds())
```

This code is used in section 51.

54. Why go through the covering sets at all, when answer 477 could be given all twenty variables at once? Because it does not finish. With every optional clause in place, the `direct` mode runs $r = 1, 2, \ldots$ on the whole of Table 2, and the cost grows more than a hundredfold a step.

⟨ Try answer 477 on all twenty variables 54 ⟩ ≡

```
in := instance{n: 20}
for _, p := range points {
    in.rows = append(in.rows, p.x)
    in.want = append(in.want, p.val)
}
for r := 1; r ≤ *maxSteps; r++ {
    start := time.Now()
    st, _, stats := solveChain(in, r)
    fmt.Printf("all twenty variables, r=%d: %v, %d megamems, %.1fs\n", r, st,
        stats.Mems/1000000, time.Since(start).Seconds())
}
```

This code is used in section 2.

55. Self-test. Two searches that report nothing but "no" need a check that they can say "yes." So I build random chains of up to five steps on five variables, using any of the ten nontrivial operations, hide a random third of the truth table, and ask both searches for the fewest steps. They must agree, and neither may exceed the chain that was built. A target that one variable already fits is left to *cheapest* alone: with nontrivial operations forced, the clauses cannot express a zero-step chain, as answer 477 warns.

```
⟨ Test the two searches against each other 55 ⟩ ≡
    rng := rand.New(rand.NewPCG(57, 2026))
    nontrivial := []uint8{1, 2, 4, 6, 7, 8, 9, 11, 13, 14}
    agreed, trivial := 0, 0
    for left := *trials; left > 0; {
        ⟨ Build a random chain and a random partial table 56 ⟩
        left--
        ex := cheapest(care, want, 5)
        if ex ≡ 0 {
            trivial++
            continue
        }
        ⟨ Find the fewest steps with the SAT solver and compare 57 ⟩
    }
    fmt.Printf("random chains: %d agreed, %d fit by a variable, of %d\n",
        agreed, trivial, *trials)
    ⟨ Test the read-once functions against random formulas 58 ⟩
```

This code is used in section 2.

56.

```
⟨ Build a random chain and a random partial table 56 ⟩ ≡
    steps := 1 + rng.IntN(5)
    nodes := []uint64{}
    for q := range 5 {
        nodes = append(nodes, varTable(q, 5))
    }
    for range steps {
        k := 1 + rng.IntN(len(nodes) - 1)
        j := rng.IntN(k)
        op := nontrivial[rng.IntN(len(nontrivial))]
        nodes = append(nodes, apply(op, nodes[j], nodes[k]) & ^uint64(0))
    }
    want := nodes[len(nodes) - 1]
    var care uint64
    for t := range 32 {
        if rng.IntN(3) ≠ 0 {
            care |= 1 ≪ t
        }
    }
    if want & care ≡ 0 ∨ want & care ≡ care {
        continue
    }
```

This code is used in section 55.

57. ⟨ Find the fewest steps with the `SAT` solver and compare 57 ⟩ ≡

```
var in instance
for t := range 32 {
    if care ≫ t & 1 ≡ 1 {
        var row uint32
        for q := range 5 {
            row |= uint32(t ≫ (4 − q) & 1) ≪ q
        }
        in.rows = append(in.rows, row)
        in.want = append(in.want, want ≫ t & 1 ≡ 1)
    }
}
in.n = 5
viaSAT := −1
for r := 1; r ≤ steps ∧ viaSAT < 0; r++ {
    if st, _, _ := solveChain(in, r); st ≡ sat.Sat {
        viaSAT = r
    }
}
if ex ≡ viaSAT ∧ ex ≤ steps {
    agreed++
} else {
    fmt.Printf("DISAGREE: built with %d steps, exhaustive %d, SAT %d\n",
        steps, ex, viaSAT)
}
```

This code is used in section 55.

58. The read-once functions get a check of their own, built a different way: put six variables in a pool, and repeatedly replace two of them by a random nontrivial combination until one is left. Complemented if need be, so that it takes $(0, \ldots, 0)$ to 0, the result must be in the list.

⟨ Test the read-once functions against random formulas 58 ⟩ ≡

```
all := readOnce(63, map[uint]map[uint64]bool{})
found := 0
for range *trials {
    pool := []uint64{}
    for q := range 6 {
        pool = append(pool, varTable(q, 6))
    }
    for len(pool) > 1 {
        a := rng.IntN(len(pool))
        b := (a + 1 + rng.IntN(len(pool) - 1)) % len(pool)
        op := nontrivial[rng.IntN(len(nontrivial))]
        v := apply(op, pool[a], pool[b])
        pool[min(a, b)] = v
        pool = slices.Delete(pool, max(a, b), max(a, b) + 1)
    }
    if pool[0] & 1 ≡ 1 {
        pool[0] = ⊕pool[0]
    }
    if all[pool[0]] {
        found++
    }
}
fmt.Printf("random read-once formulas found in the list: %d of %d\n",
    found, *trials)
```

This code is used in section 55.

59. Index.

A: 36, 37.
a: 14, 25, 32, 37, 43, 44, 58.
across: 21, 22.
acrossOK: 21, 22.
Add: 11, 13, 35.
AddClause: 41, 44, 46, 47, 48, 49.
agree: 12, 28, 52.
agreed: 55, 57.
all: 38, 58.
answer56: 18.
append: 10, 14, 17, 18, 21, 25, 26, 28, 29, 33, 34, 40, 41, 44, 45, 48, 50, 51, 54, 56, 57, 58.
apply: 32, 34, 37, 56, 58.
arctanInv: 11, 13.
B: 37.
b: 14, 23, 25, 32, 37, 58.
Background: 41.
bc: 44, 45.
big: 11, 12, 13.
bits: 14, 36, 38, 53.
bookChain: 27, 28, 29.
b2i: 22, 23, 25, 29.
care: 31, 32, 33, 34, 35, 38, 55, 56, 57.
ch: 25, 26, 41, 50, 51, 52.
chain: 24, 25, 26, 27, 41, 50.
cheapest: 33, 35, 55.
cl: 44, 45.
col: 21.
comp: 24, 25, 27, 41, 42, 46, 50, 52.
Compare: 14.
Contains: 34.
context: 41.
cost: 35.
covering: 14, 16, 38, 53.
d: 16, 32.
Delete: 58.
depth: 33.
differs: 43, 44, 45.
diffs: 14, 15, 16.
Done: 35.
down: 21, 22.
downOK: 21, 22.
e: 11, 12.
err: 41.
eval: 25, 28, 29, 52.
ex: 55, 57.
extend: 33, 34.
f: 28, 42, 44, 47, 50.
fits: 32, 33, 34, 38.
fives: 14, 15, 18, 21, 29, 35, 51.
flag: 3.
Float: 11, 12, 13.
fmt: 12, 17, 18, 21, 22, 26, 28, 29, 35, 38, 51, 52, 53, 54, 55, 57, 58.
format: 26, 28, 52.
found: 51, 52, 58.
full: 29.
gen: 14, 15, 18.
i: 21, 22, 26, 28, 32, 35, 41, 42, 44, 47, 48, 49, 50, 51.
ii: 48, 49.
in: 40, 41, 45, 46, 51, 54, 57.
instance: 39, 40, 41, 54, 57.
instanceOn: 40, 51, 53.
Int: 3, 12.
IntN: 56, 58.
j: 10, 17, 19, 24, 25, 26, 28, 29, 34, 40, 41, 42, 44, 45, 47, 48, 49, 50, 51, 52, 56.
jj: 49.
Join: 17, 18, 26.
js: 17, 40.
k: 11, 12, 13, 24, 25, 26, 34, 41, 42, 44, 45, 47, 48, 49, 50, 56.
kk: 49.
L: 34.
l: 43.
left: 55.
leftRows: 6, 10, 12.
len: 18, 25, 28, 34, 35, 38, 40, 41, 56, 58.
limit: 33.
Lit: 41, 42, 43, 44, 48.
low: 36, 37.
m: 5, 14, 16, 17, 18, 19, 21, 22, 29, 31, 35, 38, 40, 51, 52, 53.
main: 2.
make: 25, 33, 35, 42.
matches: 38.
max: 58.
maxSteps: 3, 54.
members: 14, 17, 19, 29, 40, 51, 52.
memo: 36, 37.
Mems: 51, 53, 54.
mems: 51, 53.
min: 58.

mode: 2, 3, 4, 5.
more: 34.
Mul: 11, 13.
N: 41, 42, 46, 47, 48, 49, 50.
n: 12, 24, 25, 26, 27, 39, 40, 41, 42, 45, 47, 50, 54, 57.
name: 26.
names: 18, 26, 51, 52.
New: 41, 55.
new: 11, 12, 13.
NewFloat: 11.
NewPCG: 55.
NewReplacer: 26.
NewVar: 42.
nodes: 33, 34, 56.
nontrivial: 55, 56, 58.
normalChain: 27, 28.
normalOps: 32, 34, 37.
Not: 43, 44, 46, 47, 49.
Now: 35, 38, 51, 53, 54.
o: 45.
ok: 29, 36.
on: 2, 5.
OnesCount: 36.
OnesCount32: 14, 38, 53.
op: 24, 25, 26, 32, 34, 37, 50, 56, 58.
opForm: 26.
out: 52.
P: 41, 42, 44, 46.
p: 14, 22, 28, 31, 40, 52, 54.
panic: 41.
Parse: 3.
point: 8, 9, 10.
points: 9, 10, 14, 22, 28, 31, 40, 52, 54.
pool: 58.
pow: 13.
pq: 42, 50.
prec: 11, 12, 13.
printedLeft: 20, 21.
printedRight: 20, 21, 29.
Printf: 12, 18, 21, 22, 28, 29, 35, 38, 51, 53, 54, 55, 57, 58.
Println: 18.
q: 14, 25, 29, 31, 33, 40, 52, 56, 57, 58.
quarterPi: 11, 12.
Quo: 11, 13.
r: 21, 26, 36, 37, 41, 51, 54, 57.
rand: 55.
readOnce: 36, 37, 38, 58.
Repeat: 22.
Replace: 26.
ReplaceAll: 21, 29.
rest: 36.
rightRows: 7, 10, 12.
rng: 55, 56, 58.
row: 25, 29, 40, 52, 57.
rows: 10, 12, 39, 40, 41, 45, 54, 57.
S: 36, 37.
s: 10, 12, 17, 21, 25, 26, 41, 42, 44, 46, 47, 48, 49, 50.
Sat: 41, 57.
sat: 41, 42, 43, 44, 48, 51, 53, 57.
satisfiable: 53.
satisfied: 45.
scaled: 12.
Seconds: 35, 38, 51, 53, 54.
seen: 40.
sel: 41, 42, 44, 48, 49, 50.
SetInt64: 11, 13.
SetMantExp: 12.
setName: 17, 18, 22, 29, 51.
SetPrec: 11, 12, 13.
side: 10, 12.
Since: 35, 38, 51, 53, 54.
sixes: 38, 53.
slices: 14, 34, 58.
Solve: 41.
solveChain: 41, 51, 53, 54, 57.
some: 41.
SortFunc: 14.
Sprint: 17.
Sprintf: 12, 26, 28, 51, 52.
st: 26, 41, 50, 51, 53, 54, 57.
start: 35, 38, 51, 53, 54.
stats: 51, 53, 54.
Stats: 41.
Status: 41, 51.
step: 24, 27, 50.
steps: 24, 25, 26, 27, 28, 50, 56, 57.
strings: 17, 18, 21, 22, 26, 29.
StringVar: 3.
sts: 51.
Sub: 11, 13.
sum: 13.
sync: 35.
t: 13, 19, 29, 31, 42, 44, 45, 46, 56, 57.

target: 31, 35, 38.
tenth: 29.
term: 11.
time: 35, 38, 51, 53, 54.
TrailingZeros: 36.
trials: 3, 55, 58.
trivial: 55.
tt: 22.
ttIndex: 19, 22, 31.
Unsat: 53.
used: 48.
v: 12, 25, 28, 31, 32, 34, 38, 58.
val: 8, 22, 28, 31, 40, 52, 54.
Value: 50.
varTable: 31, 33, 36, 56, 58.
viaSAT: 57.
w: 31, 32.
Wait: 35.
WaitGroup: 35.
want: 31, 32, 33, 34, 35, 38, 39, 40, 46, 54, 55, 56, 57.
wg: 35.
x: 8, 10, 13, 14, 19, 22, 28, 31, 40, 42, 44, 45, 46, 52, 54.
xnames: 28.
xx: 13.

⟨ Add the operand literals, or skip the clause 45 ⟩ Used in section 44
⟨ Build a random chain and a random partial table 56 ⟩ Used in section 55
⟨ Build the table of m and compare it with the printed ones 22 ⟩ Used in section 21
⟨ Check Table 2 against π and e 11 ⟩ Used in section 2
⟨ Check the formula and count its operations 28 ⟩ Used in section 2
⟨ Combine the read-once functions on $low \cup A$ and on the rest 37 ⟩ Used in section 36
⟨ Compare the binary expansions with the two columns 12 ⟩ Used in section 11
⟨ Compare the twelve truth tables with the answer 21 ⟩ Used in section 2
⟨ Create the variables of answer 477 42 ⟩ Used in section 41
⟨ Emit the main clauses for s_{ijk} 44 ⟩ Used in section 41
⟨ Emit the optional clauses of answers 477 and 478 47 ⟩ Used in section 41
⟨ Emit the output clauses 46 ⟩ Used in section 41
⟨ Find the covering sets 14 ⟩ Used in section 2
⟨ Find the fewest steps with the `SAT` solver and compare 57 ⟩ Used in section 55
⟨ Forbid reapplied operands and colex inversions after s_{ijk} 49 ⟩ Used in section 47
⟨ Functions 5 ⟩ Used in section 2
⟨ Global variables 4 ⟩ Used in section 2
⟨ If a chain was found, check it on Table 2 and describe it 52 ⟩ Used in section 51
⟨ Read Table 2 10 ⟩ Used in section 2
⟨ Read the chain off the solution 50 ⟩ Used in section 41
⟨ Read the command line 3 ⟩ Used in section 2
⟨ Report the covering sets 18 ⟩ Used in section 2
⟨ Rule out five steps by exhaustive search 35 ⟩ Used in section 2
⟨ Rule out five steps on the six-sets with the `SAT` solver 53 ⟩ Used in section 51
⟨ Rule out five steps, and find six, with the `SAT` solver 51 ⟩ Used in section 2
⟨ Rule out read-once formulas on six variables 38 ⟩ Used in section 35
⟨ Show how the formula fills in the tenth table 29 ⟩ Used in section 28
⟨ Step i is used by a later step 48 ⟩ Used in section 47
⟨ Test the read-once functions against random formulas 58 ⟩ Used in section 55
⟨ Test the two searches against each other 55 ⟩ Used in section 2
⟨ Try answer 477 on all twenty variables 54 ⟩ Used in section 2
⟨ Types 8 ⟩ Used in section 2

Six gates

	Section	Page
Introduction	1	1
Table 2	6	4
Covering sets	14	7
The twelve truth tables	19	9
Chains	24	11
The formula	27	12
Why five steps cannot do: the argument	30	14
Exhaustive search	31	15
The SAT route	39	19
Self-test	55	27
Index	59	30